



Product architecture audit: front-end, backend, console, and SaaS feature gaps across top-tier competitors — and a 12-point modernization blueprint

Computer

Competitive SaaS roadmap: benchmark your UI/UX, admin console, and backend stack against market leaders to close feature gaps and drive retention

Computer

Show me how to prioritize these front-end improvements

What specific backend architecture choices support this strategy

How do we structure the operations console for higher ed

I created the **Semester Product Modernization Blueprint**. It provides a 12-point modernization plan, front-end prioritization order, backend architecture choices, higher-ed operations-console structure, and phased roadmap.

The guiding strategy is not feature parity. It is to make Semester the most trusted, accessible, operationally mature, and student-useful university platform—especially where legacy portals, generic AI, and disconnected point tools struggle.

Front-end prioritization

Prioritize the user experience in this order:

P0: launch-blocking foundation

- Unified AppShell and navigation
- Real login/signup/auth-state UX
- Consistent source/freshness/privacy components
- Design tokens and shared components
- Loading, empty, error, offline, stale-data, and permission states
- Keyboard, focus, semantic, contrast, and reflow baseline
- Mobile-first core routes
- Visible autosave, recovery, and durable-save feedback

P1: retention-driving student workflows

Today as a decision workspace
Action Center
My Path and Registration Center
Schedule/conflict/backup-course workflow
Advisor Packets and controlled sharing
Contextual Ask Semester
Study Studio
Permission-aware, source-aware search

P2: faculty and institution adoption

Course Studio
Assignments, submission recovery, and receipts
Assessment and accommodation workflows
Gradebook and accessible feedback
Institution administration and policy UI
Integration-health and source-governance interface
Implementation, UAT, and training interfaces

P3: defensible differentiation

Career evidence graph
Student Credential Wallet
Campus Hub and opportunity ecosystem
Transfer, athlete, study-abroad modules
Developer portal and integration sandbox

A front-end feature cannot leave preview until it has responsive behavior, keyboard and screen-reader validation, visible focus, source/privacy states, loading/error/offline conditions, monitoring, help/support path, feature flag, and rollback plan.

Backend architecture choices

Architecture style

Start with a **modular monolith** with strict domain boundaries rather than premature microservices.

Why:
Faster delivery
Simpler transactions
Stronger tenant-aware consistency
Lower operational overhead
Easier auditability
Clear extraction path for high-scale domains later

Domain boundaries

Identity and authorization
Tenant/customer configuration
Student planning and Action Graph
Native LMS
Assessment and gradebook
Workspace/files/materials
Search and retrieval
AI Intelligence
Integrations and synchronization
Notifications
Billing and entitlements
Support/customer success
Compliance/audit
Observability/reliability

Data platform

Transactional data:
Postgres/Supabase with RLS.

Audit:
Append-only audit/event store with protected retention.

Search:
Permission-aware search index filtered by tenant, role, object, and source access.

Files:
Scoped object storage, signed access, malware/type validation, retention/deletion lifecycle.

Jobs:
Durable queue, idempotency, retries, dead-letter queue, reconciliation.

Analytics:
Separate governed aggregate model—not broad production-record browsing.

Compliance:
Controlled evidence store with version, expiry, owner, and review metadata.

Security architecture

Server-side authorization
Database RLS
Scoped storage
MFA and SSO for privileged users
Server-only secrets
Least-privilege OAuth, mTLS, LTI credentials
Feature flags and kill switches
Immutable audit events

Threat models
Automated negative authorization tests

Operations console

The Operations Console should be organized around work, not tables.

Command Center
Customers
Tenants
Identity and Access
Implementation
Support
Platform and Integrations
Reliability
Trust and Compliance
Revenue Operations
Product Quality
Audit

Academic operations modes

Registration mode:
Source freshness, registration connector health, support, release freeze.

Assessment mode:
Autosave/submission health, accommodation issues, load, PO coverage.

Grade mode:
Gradebook performance, posting, passback, reconciliation, faculty support.

Term-start mode:
SSO, roster sync, course publication, imports, onboarding, support volume.

Console role tiers

Tier 0:
Executive read-only insight.

Tier 1:
Customer success, implementation, support, finance, sales operations.

Tier 2:
Integration, SRE, security, privacy, accessibility, AI specialists.

Tier 3:
Strictly controlled platform/security/release/finance administrators.

Tier 4:
Time-limited, strongly audited, two-person break-glass access.

No role should inherit unrestricted student-record access simply because it is an internal Semester role.

12 modernization priorities

1. Unified AppShell and context-first UX.
2. Today as a decision workspace.
3. Decision Packets for registration, advising, study, career, and study abroad.
4. Contextual Ask Semester.
5. Accessible native LMS advantage.
6. Integration Control Plane.
7. Source-of-truth and data-lineage engine.
8. Policy-as-code.
9. High-assurance tenant and role architecture.
10. Academic-critical reliability.
11. Institution Launch Method and customer operating platform.
12. Student portability through Credential Wallet and evidence graph.

Roadmap

Wave 0:

Assess, stabilize, separate demo/production, fix PO gaps.

Wave 1:

AppShell, real auth/RLS, core student planning, source/privacy states, monitoring and recovery.

Wave 2:

SSO/SCIM, Integration Control Plane, trust package, Launch Center, customer portal, policy-as-code.

Wave 3:

Native LMS, assessment/gradebook, LTI/OneRoster/QTI/Common Cartridge, governed AI, migration/parallel run.

Wave 4:

Credential Wallet, evidence graph, CASE/Open Badges/CLR, career/campus ecosystem, developer platform, research benchmarks.

Final decision rule

Prioritize any work that does at least two of the following:

- Makes a recurring student decision clearer.
- Reduces enterprise procurement risk.
- Unblocks multiple modules.
- Prevents data or work loss.
- Improves critical accessibility.
- Makes implementation repeatable.
- Improves registration/finals reliability.
- Creates portable student value.

Reduces support burden.
Creates evidence for a defensible market claim.

Defer anything that is decorative, duplicates an authoritative system without a clear advantage, depends on surveillance, lacks a source/privacy model, or cannot be reliably supported.